

`router/index.js` ファイルと関連コンポーネント

これまではemitでのコンポーネント間のデータ通信をしていましたが、多数の画面間のトラフィックを整理する必要があるため、ルーティング設定をまとめたファイルの新設する必要があります。

各コンポーネントとも、新設されたルーティング設定を反映していく必要があります。

まずはログイン、ユーザー新規登録、ダッシュボードの各画面分のコンポーネントを修正します。

新設ファイル

router/index.js ファイルを新設します。

```
import { createRouter, createWebHistory } from 'vue-router';
import LoginForm from '../components/LoginForm.vue';
import RegistForm from '../components/RegisterForm.vue';
import AttendanceBoard from '../components/AttendanceBoard.vue';
import DashboardView from '../components/DashboardView.vue'; // ← 追加

const routes = [
  { path: '/login', component: LoginForm },
  { path: '/register', component: RegistForm },
  { path: '/attendance', component: AttendanceBoard },
  { path: '/dashboard', component: DashboardView }, // ← 追加
  { path: '/', redirect: '/login' } // 初期アクセスはログインへ
];
```

```
const router = createRouter({
  history: createWebHistory(),
  routes
});

export default router;
```

修正ファイル

main.js の修正

```
import { createApp } from 'vue'
import './style.css'
import App from './App.vue'
import router from './router' // 1. 作成したルーター設定（router/index.jsなど）を読み込む

createApp(App).mount('#app')

const app = createApp(App)
```

```
// 2. アプリ全体にルーターを登録する（有効化する）  
// これを挟むことで、App.vue の中で <router-view> や <router-link> が使えるようになります。  
app.use(router)  
  
// 3. 最後に画面にマウントする  
app.mount('#app')
```

Dashboard.vue の修正

```
<template>  
  <div class="dashboard-container">  
    <header class="dashboard-header">  
      <h1>ダッシュボード</h1>  
      <span class="user-welcome">ようこそ、{{ userName }} さん</span>  
    </header>  
  </div>  
</template>
```

```
<section class="announcement-section">
  <h2>お知らせ・通知</h2>
  <div v-if="announcements.length === 0" class="no-announcement">
    現在新しいお知らせはありません。
  </div>

  <div v-else class="announcement-list">
    <div v-for="item in announcements" :key="item.announcementId" class="announcement-card">
      <h3>{{ item.announcementTitle }}</h3>
      <p>{{ item.announcementAbout }}</p>
    </div>
  </div>
</section>

<section class="main-actions">
  <button @click="navigateTo('/attendance')" class="btn-main btn-attendance">
    打刻画面へ移動
  </button>
  <button @click="handleLogout" class="btn-main btn-logout">
    ログアウト
  </button>
</section>
```

```
<section class="menu-section">
  <h2>各種メニュー</h2>
  <div class="menu-grid">
    <button @click="navigateTo('/schedule-demand')" class="btn-menu">
      予定申請
    </button>
    <button @click="navigateTo('/timechange-demand')" class="btn-menu">
      打刻内容編集申請
    </button>
    <button @click="navigateTo('/profile-edit')" class="btn-menu">
      アカウント情報編集
    </button>

    <button v-if="isAuth === 1" @click="navigateTo('/admin')" class="btn-menu btn-admin">
      【管理者】各種管理画面
    </button>
  </div>
</section>
</div>
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
// import apiClient from '../api/axios'; // 既存の共通APIクライアント
import apiClient from '../api.js';

// 親コンポーネントやログイン状態から受け取るプロパティ（仮受け）
const props = defineProps({
  accountId: { type: String, default: '' },
  userName: { type: String, default: 'サンプルユーザー' },
  isAuth: { type: Number, default: 0 } // 0:一般, 1:管理者
});

const router = useRouter();
const announcements = ref([]);

// お知らせデータの取得
const fetchAnnouncements = async () => {
  try {
    const response = await apiClient.get('/dashboard/announcements');
    announcements.value = response.data;
  } catch (error) {
    console.error('お知らせの取得に失敗しました', error);
  }
};
```

```
// 画面遷移ハンドラー
const navigateTo = (path) => {
  router.push(path);
};

// ログアウト処理
const handleLogout = () => {
  if (confirm('ログアウトしますか?')) {
    // セッションやトークンのクリア処理をここに記述
    localStorage.removeItem('token'); // 例
    router.push('/login');
  }
};

onMounted(() => {
  fetchAnnouncements();
});
</script>

<style scoped>
.dashboard-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 20px 20px 20px; /* ヘッダー等との兼ね合い */
  font-family: sans-serif;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 2px solid #34495e;
  padding-bottom: 10px;
  margin-bottom: 20px;
}
```



```
/* 上段: お知らせスタイル */
.announcement-section {
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 8px;
  border-left: 5px solid #3498db;
  margin-bottom: 25px;
}
.announcement-card {
  background: white;
  padding: 10px 15px;
  margin-top: 10px;
  border-radius: 4px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}

/* 中段: メインアクション */
.main-actions {
  display: flex;
  gap: 15px;
  margin-bottom: 30px;
}
.btn-main {
  flex: 1;
  padding: 15px;
  font-size: 1.2rem;
  font-weight: bold;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s;
}
.btn-attendance { background-color: #2ecc71; color: white; }
.btn-attendance:hover { background-color: #27ae60; }
.btn-logout { background-color: #95a5a6; color: white; }
.btn-logout:hover { background-color: #7f8c8d; }
```

```
/* 下段: メニューグリッド */
.menu-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 15px;
  margin-top: 15px;
}
.btn-menu {
  padding: 20px;
  font-size: 1rem;
  background-color: #34495e;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s;
}
.btn-menu:hover { background-color: #2c3e50; transform: translateY(-2px); }
.btn-admin { background-color: #e67e22; }
.btn-admin:hover { background-color: #d35400; }
</style>
```

RegisterForm.vue の修正

```
<template>
  <div class="register-container">
    <div class="register-card">
      <h2 class="form-title">ユーザー登録</h2>

      <form @submit.prevent="handleRegister" class="registration-form">
        <div class="form-group">
          <label for="userName">ユーザー名</label>
          <input id="userName" v-model="userName" type="text" :disabled="isRedirecting" required placeholder="例：田中 太郎" />
        </div>

        <div class="form-group">
          <label for="userId">ユーザーID (メールアドレス)</label>
          <input id="userId" v-model="userId" type="email" :disabled="isRedirecting" required placeholder="example@mail.com" />
        </div>

        <div class="form-group">
          <label for="password">パスワード</label>
          <input id="password" v-model="password" type="password" :disabled="isRedirecting" required />
        </div>

        <div class="form-group">
          <label for="passwordConfirm">パスワード (確認用)</label>
          <input id="passwordConfirm" v-model="passwordConfirm" type="password" :disabled="isRedirecting" required />
        </div>
      </form>
    </div>
  </div>
</template>
```

```

<div v-if="errorMessage" class="error-message">{{ errorMessage }}</div>

<div v-if="successMessage" class="success-message">{{ successMessage }}</div>

<div class="button-group">
  <button type="submit" class="submit-btn" :disabled="isRedirecting">これで登録する</button>
  <button type="button" @click="resetForm" class="reset-btn" :disabled="isRedirecting">リセット</button>
</div>
</form>

<div class="footer-link">
  <span @click="!isRedirecting && router.push('/login')" class="login-link" :class="{ 'disabled-link': isRedirecting }">
    既にアカウントをお持ちの方はこちら（ログイン画面へ）
  </span>
</div>
</div>
</div>
</template>

<script setup>
import { ref } from 'vue';
import { useRouter } from 'vue-router';

const router = useRouter();

const userName = ref('');
const userId = ref('');
const password = ref('');
const passwordConfirm = ref('');
const errorMessage = ref('');

// ★新しく追加する状態変数
const successMessage = ref(''); // 成功メッセージ用
const isRedirecting = ref(false); // 3秒間のリダイレクト中かどうかを判定

```

```

const emit = defineEmits(['register-success', 'switch-to-login']);

const handleRegister = async () => {
  // 1. パスワード一致チェック
  if (password.value !== passwordConfirm.value) {
    errorMessage.value = "パスワードが一致しません。";
    return;
  }

  try {
    const response = await fetch('https://ubiquitous-spork-4vq65g5rr79c5j6q-8080.app.github.dev/api/users/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        userName: userName.value,
        userId: userId.value,
        password: password.value
      })
    });

    if (response.ok) {
      errorMessage.value = ""; // エラーを消す
      isRedirecting.value = true; // 連打や入力を防ぐため無効化モードオン

      // ★ カウントダウンタイマー（3秒）の作成
      let countdown = 3;
      successMessage.value = `登録が完了しました！${countdown}秒後にログイン画面へ移動します...`;

      const timer = setInterval(() => {
        countdown -= 1;
        if (countdown > 0) {
          successMessage.value = `登録が完了しました！${countdown}秒後にログイン画面へ移動します...`;
        } else {
          clearInterval(timer); // タイマーを止める

          // 既存のイベント通知を送りつつ、ルーターでログイン画面へジャンプ
          emit('switch-to-login');
          router.push('/login');
        }
      }, 1000); // 1秒ごとに実行
    }
  }
}

```

```
    } else {
      const errorText = await response.text();
      errorMessage.value = errorText || "登録に失敗しました。";
    }
  } catch (error) {
    errorMessage.value = "サーバーとの通信に失敗しました。";
  }
};
```

```
const resetForm = () => {
  userName.value = '';
  userId.value = '';
  password.value = '';
  passwordConfirm.value = '';
  errorMessage.value = '';
  successMessage.value = '';
};
</script>
```

```
<style scoped>
/* 既存のスタイルに以下を追加、または一部書き換え */
```

```
.register-container {
  padding-top: 40px;
  display: flex;
  justify-content: center;
}
```

```
.register-card {
  width: 100%;
  max-width: 400px;
  padding: 20px;
}
```

```
.form-title {
  text-align: center;
  margin-bottom: 30px;
}
```

```
.form-group {
  margin-bottom: 20px;
  display: flex;
  flex-direction: column;
  text-align: left;
}

.form-group label {
  margin-bottom: 8px;
  font-weight: bold;
}

.form-group input {
  padding: 10px;
  border-radius: 4px;
  border: 1px solid #ccc;
  font-size: 1rem;
}

/* 入力不可状態（リダイレクト中）の入力欄の見た目 */
.form-group input:disabled {
  background-color: #f5f5f5;
  color: #999;
  cursor: not-allowed;
}
```

```
.error-message {
  color: #ff4444;
  background-color: #fdf2f2; /* 成功メッセージに合わせて薄い背景色を追加して視認性を向上 */
  padding: 6px 10px; /* 上下の余白を小さく（15px → 6px）してコンパクトに */
  border: 1px solid #fde8e8; /* ほんのり枠線を付与 */
  border-radius: 4px;
  margin-bottom: 15px;
  text-align: center;
  font-size: 0.85rem; /* 文字サイズを少し小さく（1rem → 0.85rem）して折り返しを防止 */
}

/* 登録成功メッセージのスタイル（安心感を与える緑系） */
.success-message {
  color: #27ae60;
  background-color: #e8f8f5;
  padding: 6px 10px; /* 上下の余白を小さく（10px → 6px）してスマートに */
  border: 1px solid #2ecc71;
  border-radius: 4px;
  margin-bottom: 15px;
  text-align: center;
  font-weight: bold;
  font-size: 0.85rem; /* 文字サイズを少し小さくして「〇秒後に～」が1行に収まりやすく調整 */
}

.button-group {
  margin-top: 30px;
  display: flex;
  gap: 15px;
  justify-content: center;
}
```



```
.submit-btn {
  background-color: #007bff;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 4px;
  cursor: pointer;
  flex: 1;
}

.reset-btn {
  background-color: #6c757d;
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: 4px;
  cursor: pointer;
  flex: 1;
}

/* ボタンが無効化されているときのスタイル */
button:disabled {
  background-color: #cccccc !important;
  color: #888888 !important;
  cursor: not-allowed;
  filter: none !important;
}
```

```
.footer-link {  
  margin-top: 25px;  
  text-align: center;  
}  
  
.login-link {  
  color: #007bff;  
  text-decoration: underline;  
  cursor: pointer;  
  font-size: 0.9rem;  
}  
  
/* リダイレクト中にリンクをクリックできないようにする制御 */  
.disabled-link {  
  color: #999999 !important;  
  text-decoration: none;  
  cursor: not-allowed;  
}  
  
.submit-btn:hover, .login-link:hover {  
  filter: brightness(1.2);  
}  
</style>
```

LoginForm.vue の修正

```
<template>
  <div class="login-container">
    <div class="login-card">
      <h2 class="form-title">ログイン</h2>

      <form @submit.prevent="handleLogin" class="login-form">
        <div class="form-group">
          <label for="userId">ユーザーID</label>
          <input id="userId" v-model="userId" type="text" required placeholder="example@mail.com" />
        </div>

        <div class="form-group">
          <label for="password">パスワード</label>
          <input id="password" v-model="password" type="password" required />
        </div>

        <div v-if="errorMessage" class="error-message">{{ errorMessage }}</div>

        <div class="button-group">
          <button type="submit" class="submit-btn">ログイン</button>
        </div>
      </form>
    </div>
  </div>
</template>
```

```

    <div class="footer-link">
      <span @click="router.push('/register')" class="register-link">
        アカウントをお持ちでないですか？（新規ユーザー登録へ）
      </span>
    </div>
  </div>
</div>
</template>

<script setup>
import { ref } from 'vue';
import apiClient from '../api';
import { useRouter } from 'vue-router'; // ★重要: ルーターをインポート

const router = useRouter(); // ★重要: ルーターオブジェクトを取得

const userId = ref('');
const password = ref('');
const errorMessage = ref('');

const emit = defineEmits(['login-success']);

// ログイン処理（router導入前）
// const handleLogin = async () => {
//   try {
//     const response = await apiClient.post('/users/login', {
//       userId: userId.value,
//       password: password.value
//     });
//
//     // 親コンポーネントにイベントを送る
//     emit('login-success', response.data);
//   } catch (error) {
//     console.error('ログイン失敗:', error);
//     errorMessage.value = 'ユーザーIDまたはパスワードが正しくありません。';
//   }
// };

```

```
// ログイン処理（router導入後）
const handleLogin = async () => {
  try {
    const response = await apiClient.post('/users/login', {
      userId: userId.value,
      password: password.value
    });

    // 1. ローカルストレージ等に、バックエンドから返ってきたユーザー情報を保存する（後で認証ガードに使うため）
    // ※ response.data の構造（accountId や userName が入っているか）に合わせて調整してください
    localStorage.setItem('user', JSON.stringify(response.data));

    // 2. 親へのemit（もし動かなくても保険として残す、不要なら消してもOK）
    emit('login-success', response.data);

    // 3. ★ここで直接、新設したダッシュボード画面へジャンプさせる！
    router.push('/dashboard');

  } catch (error) {
    console.error('ログイン失敗:', error);
    errorMessage.value = 'ユーザーIDまたはパスワードが正しくありません。';
  }
};
</script>

<style scoped>
/* コンテナ全体の余白 */
.login-container {
  padding-top: 40px; /* 見出し上部のスペース */
  display: flex;
  justify-content: center;
}

.login-card {
  width: 100%;
  max-width: 400px;
  padding: 20px;
}
```

```
.form-title {
  text-align: center;
  margin-bottom: 30px;
}

.form-group {
  margin-bottom: 20px; /* 上下の間隔を確保 */
  display: flex;
  flex-direction: column; /* ラベルを上、入力欄を下に配置 */
  text-align: left; /* 左寄せにしてフォームらしさを強調 */
}

/* ラベルのスタイルをRegisterFormの太文字・余白に統一 */
.form-group label {
  margin-bottom: 8px;
  font-weight: bold;
}

/* 入力インプットのサイズ、パディング、境界線を統一 */
.form-group input {
  padding: 10px;
  border-radius: 4px;
  border: 1px solid #ccc;
  font-size: 1rem;
}

.error-message {
  color: #ff4444;
  margin-bottom: 15px;
  text-align: center;
}
```

```
/* ボタンエリアの設定（横幅100%ではなく、横に並んでも自然な幅に制限） */
.button-group {
  margin-top: 30px; /* フォームとの間隔 */
  display: flex;
  justify-content: center;
}

/* ログインボタン：RegisterFormの「これで登録する」と同色にしつつ、サイズを「小さく自然な大きさ」に調整 */
.submit-btn {
  background-color: #007bff; /* RegisterFormのボタンと同色の鮮やかな青 */
  color: white;
  border: none;
  padding: 10px 40px; /* 横のパディングを広げて存在感を適正化 */
  border-radius: 4px;
  cursor: pointer;
  font-size: 1rem;
  font-weight: bold;
  min-width: 150px; /* 小さすぎず、大きすぎない自然な横幅 */
  transition: filter 0.2s;
}

/* 下部リンクの調整：RegisterFormのfooter-linkとクラス、カラー、ホバーエフェクトを完全統一 */
.footer-link {
  margin-top: 25px;
  text-align: center;
}

.register-link {
  color: #007bff; /* 視認性を高めるため、登録画面と同色の青系に設定 */
  text-decoration: underline;
  cursor: pointer;
  font-size: 0.9rem;
}

.submit-btn:hover, .register-link:hover {
  filter: brightness(1.2); /* ホバー時に少し明るくして反応を示す */
}
</style>
```

App.vue の修正

```
<template>
  <div id="app">
    <AppHeader v-if="showHeader" />

    <main class="main-content" :class="{ 'has-header': showHeader }">
      <router-view />
    </main>
  </div>
</template>

<script setup>
import { ref } from 'vue';
import { computed } from 'vue';
import { useRoute } from 'vue-router';
import AppHeader from './components/AppHeader.vue';
import LoginForm from './components/LoginForm.vue';
import AttendanceBoard from './components/AttendanceBoard.vue';
import RegisterForm from './components/RegisterForm.vue';

// ログイン前（初期・登録）か、ログイン後（打刻・ダッシュボード）かをURLパスで判定
const showHeader = computed(() => {
  // window.location.pathname を使うことで、ルーターの初期化前でも確実に現在のURLを取得します
  const currentPath = window.location.pathname;

  // ルートパス「/」はログイン画面にリダイレクトされる想定のため、
  // 「/」「/login」「/register」の3つのときはヘッダーを「非表示（false）」にします
  if (currentPath === '/' || currentPath === '/login' || currentPath === '/register') {
    return false;
  }

  // それ以外の画面（/dashboard や /attendance など）では「表示（true）」にします
  return true;
});
</script>
```



```
<style scoped>
/* ヘッダー固定に伴い、コンテンツが下に隠れないように余白を作る */
.main-content.has-header {
  padding-top: 80px;
}

.app-header { display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px 20px;
  background: #333; color: white;
}
.hamburger { background: none; border: none; cursor: pointer; display: flex; flex-direction: column; gap: 5px; }
.hamburger span { display: block; width: 25px; height: 3px; background: white; }
.menu-container { position: relative; }
.dropdown-menu {
  position: absolute;
  right: 0; top: 40px;
  background: white;
  color: #333;
  border: 1px solid #ddd; border-radius: 5px;
  padding: 10px;
  min-width: 150px;
  z-index: 100;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}
```

```
.logout-btn { width: 100%;  
  padding: 8px;  
  background: #f44336; color: white;  
  border: none; border-radius: 3px;  
  cursor: pointer;  
}  
.user-info { font-size: 0.9rem; margin: 5px 0; }  
.switch-mode-link {  
  margin-top: 30px; /* メッセージとの間隔を広げる */  
  text-align: center;  
}  
.text-button {  
  background-color: #007bff; /* ログインボタンと同色（青系） */  
  color: white;  
  border: none;  
  border-radius: 4px;  
  padding: 10px 20px; /* ボタンを大きく */  
  font-size: 1rem;  
  cursor: pointer; /* マウスカーソルを指マークに */  
  transition: background-color 0.3s;  
}  
.text-button:hover {  
  background-color: #0056b3; /* ホバー時に少し暗く */  
}  
</style>
```